

E-graphs modulo theories

presented by Sofia Brookie

E-graph refresher: congruence closure

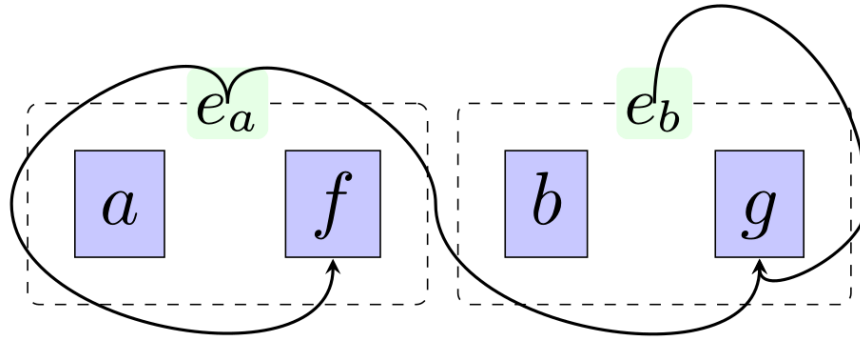


Figure 1:

$$f(a) = a$$

$$g(f(f(a)), b) = g(a, g(a, b))$$

$$g(f(a), b) = b$$

E-graph refresher: computing congruence closure

- Keep track of equivalence classes (E-classes) with a union-find

E-graph refresher: computing congruence closure

- Keep track of equivalence classes (E-classes) with a union-find
- Rather than representing terms, represent E-nodes (e.g. $f(e_1, e_2)$)

E-graph refresher: computing congruence closure

To insert a term: flatten it leaf-to-root, looking up or inserting an equivalence class for each E-node

E-graph refresher: computing congruence closure

To merge terms: call `union()` on the union-find

E-graph refresher: computing congruence closure

To compute the congruence closure, we have to do `rebuild()`

E-graph refresher: computing congruence closure

To compute the congruence closure, we have to do rebuild()

1. For every E-node $f(e_1, e_2, \dots)$, update it to $f(\text{find}(e_1), \text{find}(e_2))$

E-graph refresher: computing congruence closure

To compute the congruence closure, we have to do rebuild()

1. For every E-node $f(e_1, e_2, \dots)$, update it to $f(\text{find}(e_1), \text{find}(e_2))$
2. Whenever we have $f(e_1, e_2, \dots) \in e_j$ and also $\in e_k$, call $\text{union}(e_j, e_k)$

E-graph refresher: computing congruence closure

To compute the congruence closure, we have to do rebuild()

1. For every E-node $f(e_1, e_2, \dots)$, update it to $f(\text{find}(e_1), \text{find}(e_2))$
2. Whenever we have $f(e_1, e_2, \dots) \in e_j$ and also $\in e_k$, call $\text{union}(e_j, e_k)$

A bit slow (maybe). But we can defer rebuilding and/or use Downey-Sethi-Tarjan (aka binarizing the function symbols)

E-graph refresher: equality saturation

Congruence closure decides the theory of ground equations. What about identities?

E-graph refresher: equality saturation

Idea: if we have $\forall x..., t = s$, try instantiating the quantifier with known E-classes.

Surely if we have some equational proof we want to find, it will involve identities instantiated at ground terms in this obvious way.

E-graph refresher: equality saturation

Idea: if we have $\forall x..., t = s$, try instantiating the quantifier with known E-classes.

Surely if we have some equational proof we want to find, it will involve identities instantiated at ground terms in this obvious way.

The truth is a little more subtle, but I will gloss over this.

Equality saturation

After instantiation, we have a ground equation $t' = s'$.

We can just insert this into the graph and run congruence closure/
rebuilding.

Equality saturation

Is this complete? Not always. Another complicated story...

Equality saturation

Is this complete? Not always. Another complicated story...

But a key takeaway here is that if t' and s' are already represented, equating them *shrinks* the E-graph. But inserting new things can grow the E-graph arbitrarily.

E-graphs vs AC

The bane of equality saturation is operators that are AC or similar (ACU, abelian groups, etc.).

For simplicity, we will use ‘+’ as a generic name for an AC operator.

E-graphs vs AC

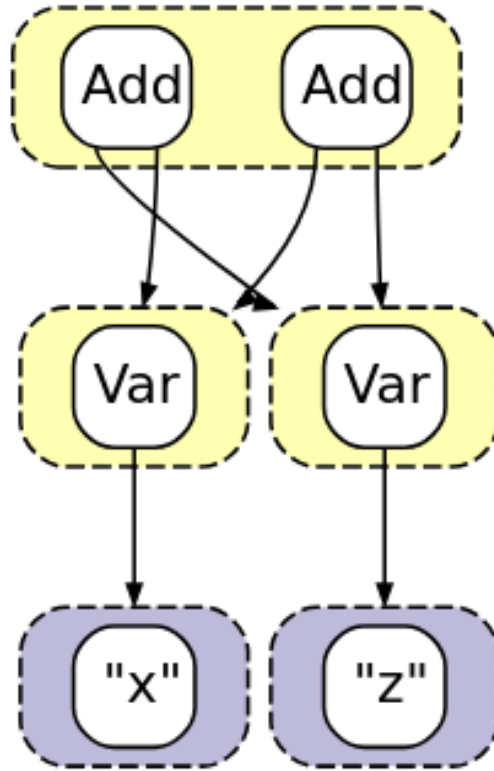


Figure 2: Size 2 (saturation)

E-graphs vs AC

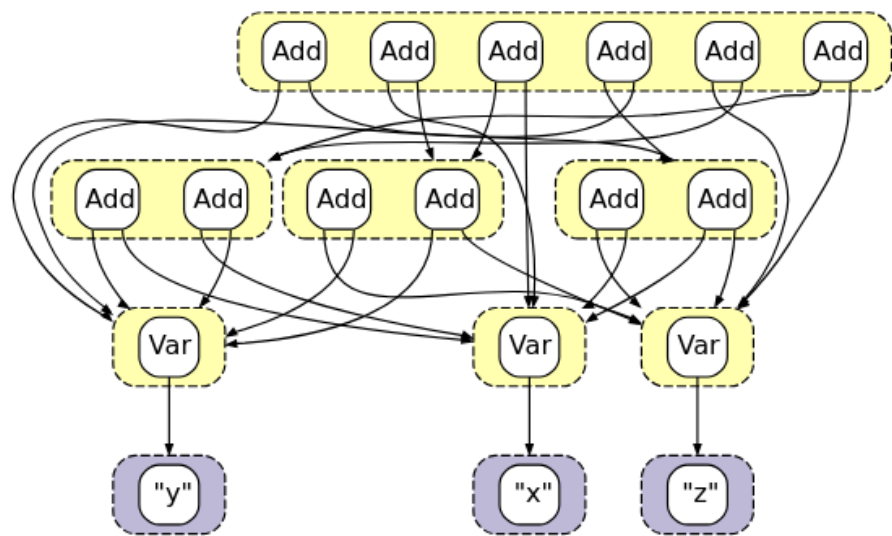


Figure 3: Size 3 (saturation)

E-graphs vs AC

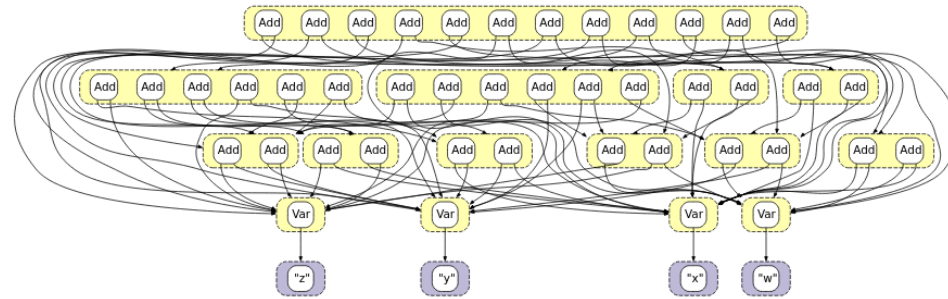


Figure 4: Size 4 (timeout)

E-graphs vs AC

Blindly saturating with A and C gives $O(2^n)$ E-classes if you have a starting term with n +s ($a + b + c + \dots$), and each E-class has $O(2^n)$ E-nodes in it! So, **doubly exponential**.

What about just A ?

A is ‘only’ cubic if you have no ‘loops’. Otherwise, it is $O(\infty)$. But making progress here is hard, given that A is fundamentally undecidable.

E-graphs vs AC: AC + other stuff

Consider an affine function f with identity

$$\forall xy. f(x + y) + 1 = f(x) + f(y)$$

and the starting term $f(a) + f(b) + f(c)$. We can to rewrite this to $f(a + b + c) + 2$

E-graphs vs AC

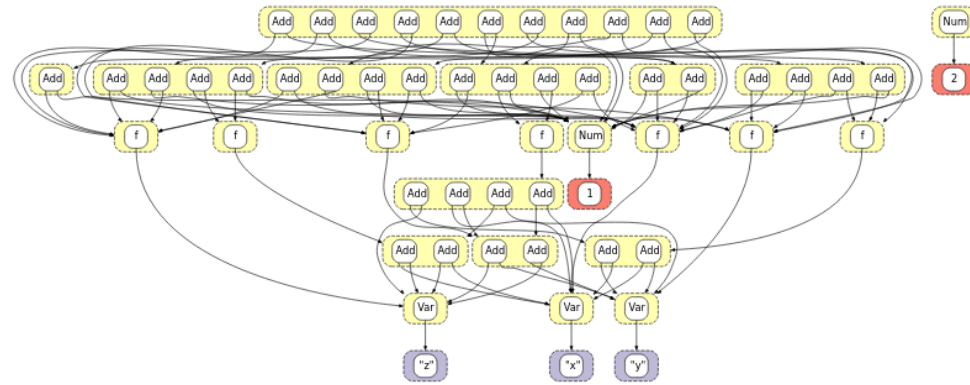


Figure 5: Size 4 (timeout)

The solution? EMTs

The SAT people have added theory-specific solvers and made very successful SMT solvers. Why can't we handle AC in a specific way, rather than relying on equality saturation?

The solution? EMTs

The SAT people have added theory-specific solvers and made very successful SMT solvers. Why can't we handle AC in a specific way, rather than relying on equality saturation?

EMTs seems to be an idea thrown around for a while, in the hope that someone will eventually make this natural thing work. Unfortunately, it is not as easy as has been suggested.

EMT example

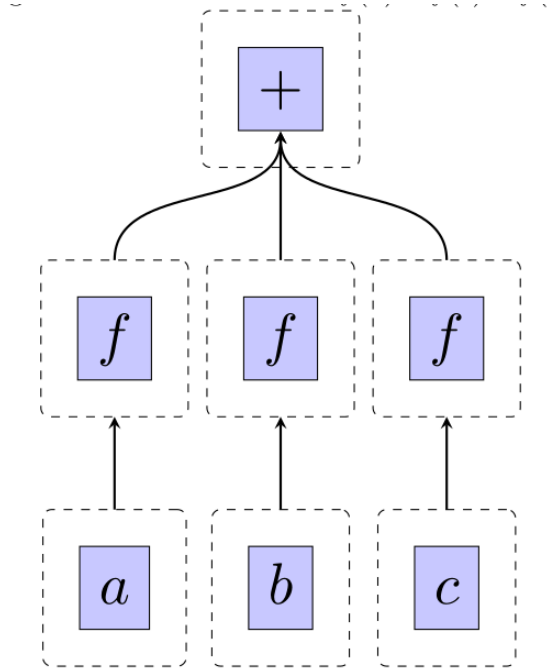


Figure 6: EMTs to the rescue...

EMT example

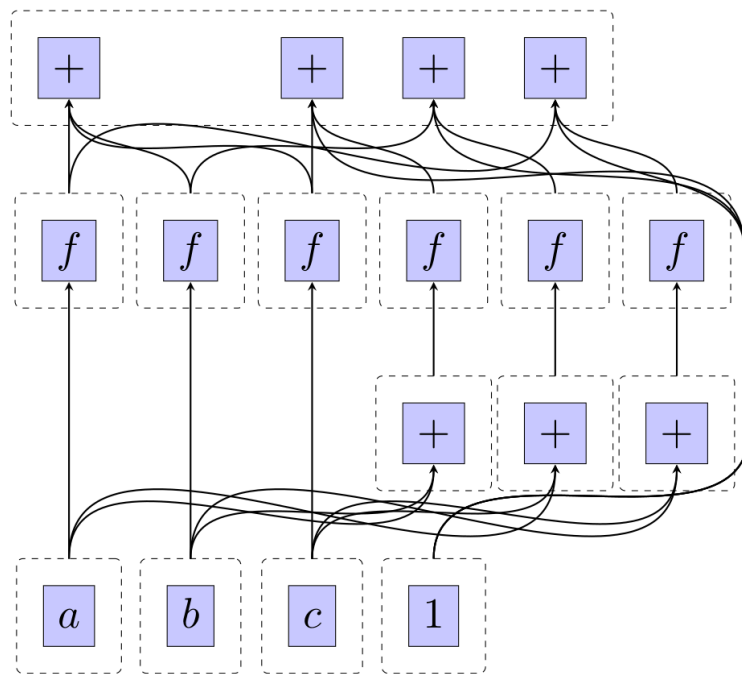


Figure 7: EMTs to the rescue...

The solution??

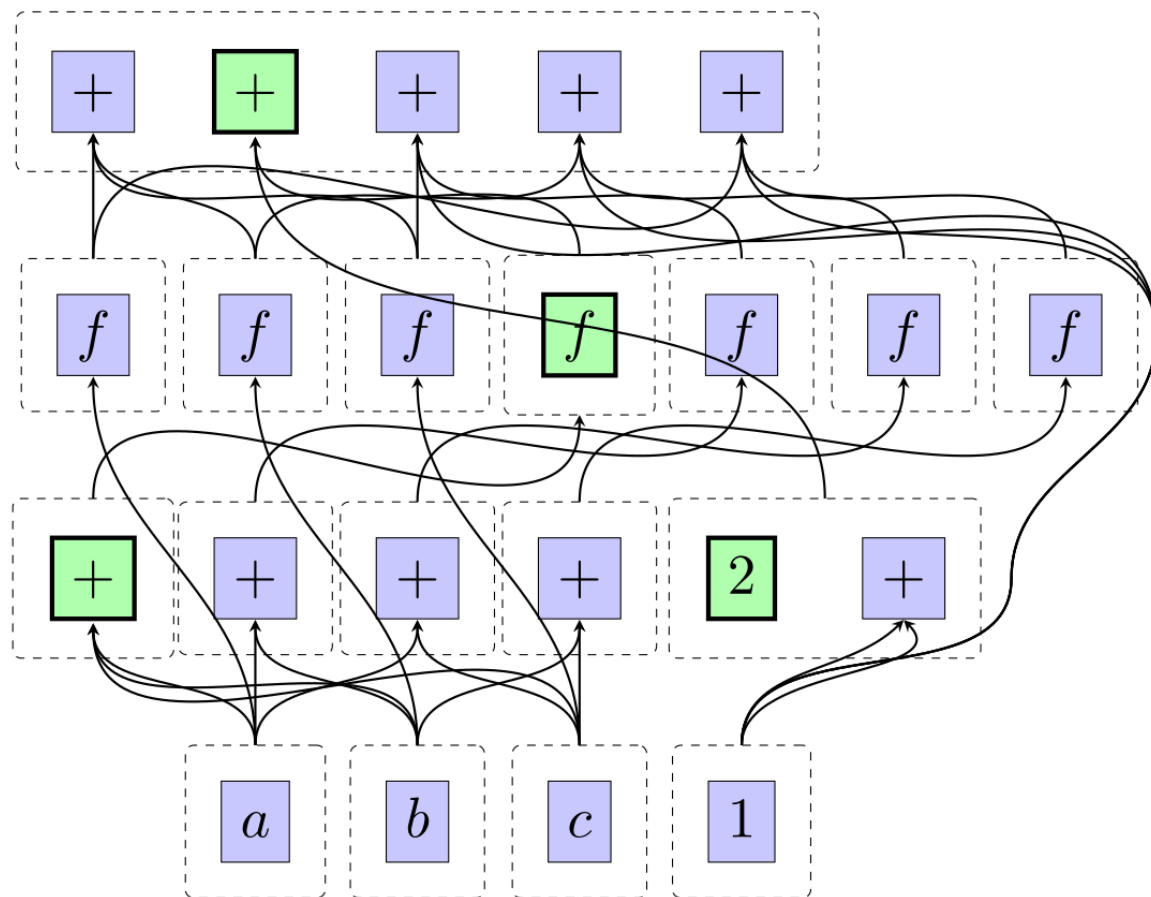


Figure 8: EMTs to the rescue...

E-graphs as rewrite systems

E-graphs connect very closely to the theory of **rewriting**.

The ‘Abstract Congruence Closure’ perspective is that E-graphs are building a *convergent term rewriting system* on an extended signature.

Congruence closure is very similar to ‘ground Knuth-Bendix completion’ from rewriting.

(Equality saturation is a bit of a stranger animal but still vaguely completion-flavoured)

E-graphs as tree automata

E-graphs are reachable deterministic bottom-up tree automata

- Reachable: every E-class is inhabited by some E-node
- Deterministic: if $f(s_1, s_2) \rightarrow s_3$ and $f(s_1, s_2) \rightarrow s_4$ then $s_3 = s_4$
- Bottom-up: there is a notion of top-down deterministic tree automata but it is weaker than deterministic bottom-up (compare the string automata case, where left-to-right and right-to-left are equivalent)

Inspirations for adding the “mod T”

Rewriting modulo theories

Algebraic automata (that is, tree automata on non-free theories)

The neat thing about E-graphs

...is that they add a new symbol for everything. $f(e_1)$? Well, if we haven't seen that before, give it a new E-class/tree-automata state

Confluence

If we have a term that is ‘in multiple E-classes’, we ought to merge the E-classes.

This is what rebuilding does:

$e_1 \rightarrow e_2$ and $f(\dots e_1 \dots) \rightarrow e_3$ (‘parent-child overlap’)

$f(\dots e_1 \dots) \rightarrow e_3$ and $f(\dots e_1 \dots) \rightarrow e_4$ (‘self-overlap’)

The new symbols for E-classes means we can handle granchild overlap, etc, by iterating these basic steps.

...which is how they simplify confluence

If we have a critical pair of terms, they must have some relevant ‘overlap’ between rules. Any overlap must be a case of subterm containment. Every subterm gets a new symbol, so we only have to check for a few cases:

$e_1 \rightarrow e_2$ and $f(\dots e_1 \dots) \rightarrow e_3$ (‘parent-child overlap’)

$f(\dots e_1 \dots) \rightarrow e_3$ and $f(\dots e_1 \dots) \rightarrow e_4$ (‘self-overlap’)

Grandchild overlap, etc, is handled by fixpoint propagation.

Key notion: critical pairs

A critical pair is some minimal term that can rewrite in two ways/is in two E-classes. To do congruence closure (modulo T) properly, we need to find critical pairs and resolve them, by merging the E-classes.

AC-critical pairs

$$(a + b) + c = e_1$$

$$b + (c + d) = e_2$$

$b + c$ is the *overlap* of an AC-critical pair:

$$e_1 + d \stackrel{AC}{=} a + b + c + d \stackrel{AC}{=} a + e_2$$

What are we doing in AC equality saturation? Well, any of the $O(2^n)$ AC-subterms *could* become an AC-critical pair. We are eagerly giving new E-classes to them. Just in case. After all, equality saturation only

works by grounding identities: we have not given a mechanism to ground something only when it's actually needed.

EMTs and the word problem

Any mechanism to automatically resolve all T -critical pairs is a solver for the T -word problem.

(Resolving critical pairs = scanning for implied equalities)

EMTs, prehistorically

Existing suggestions for EM(T)s fall along two very similar lines:

- Canonize the T parts of the E-graph, and work mod T in some way
- Treat E-nodes as ‘containers’ (of E-class names), and move from fixed-arity function symbols to lists, multisets/bags, etc

Either way is an attempt to bake the theories in:

- $A(U)$ = semigroups = lists = associate to the right
- $AC(U)$ = commutative semigroups = multisets/bags = sort ascending and associate to the right

EMTs, the end of prehistory

The intuition:

‘Why do we need all these AC-equivalent nodes? Can’t we just pick an AC-representative?’

Yes, *but*

You need some method of finding T -critical pairs, and to make a complete algorithm, you must *eventually* resolve every pair. And also match modulo T to instantiate identities.

Counterexamples to prehistoric EMTs

$$a + b = a$$

Modulo A, we ought to generate the **infinite** set of implied subterms $b, b + b, \dots$. So, prehistoric methods fail when E-graphs are **not flat terms**.

Counterexamples to prehistoric EMTs

$$b + a + c = a$$

The infinite set of equivalent terms modulo A is now $nb + a + nc$. We can no longer recognize the set of all re-associations of this, or the set of canonized forms of this, with a tree automata.

Counterexamples to prehistoric EMTs

$$a + (b + c) = d$$

$$a + b = e$$

$$e + c = f$$

Prehistoric methods might canonize, but this means they don't scan for implied equalities among a set of (canonized!) equations. Actually...

A and AC

Finding A-critical pairs, and matching modulo A, is quite feasible. Still, there is not ‘packaged’ solution possible: the A-word problem is undecidable. So, the best we can do is just provide a list of critical pairs for the ‘outer loop’ of equality saturation to choose to resolve (or not)

AC is better: the word problem is decidable (Mayr-Meyer). It’s ESPACE-complete... Sounds bad, but E-graphs are EESPACE space here, but more importantly: we can solve common cases of the AC word problem faster than ESPACE (hopefully), whereas E-graphs *must* chug along to

saturation to be *certain* (as they have no smart way of knowing if some AC-subterm is relevant to the critical pair problem)

The AC word problem, a la Bachmair et al.

Given $a_1 + a_2 + \dots \rightarrow e_1$ and $b_1 + b_2 + \dots \rightarrow e_2$, construct

$$e_1 + \dots = a_1 + a_2 + \dots + b_1 + b_2 = \dots + e_2$$

Orient it by some (multiset) ordering and add it as a rule.

The AC word problem, the polynomial encoding

If $abc = def$, encode this as a polynomial equation $x_a x_b x_c - x_d x_e x_f = 0$.

This is not overkill, since we have the ESPACE lower bound...

The AC word problem, the polynomial encoding

If $abc = def$, encode this as a polynomial equation $x_a x_b x_c - x_d x_e x_f = 0$.

This is not overkill, since we have the ESPACE lower bound...

A set of AC equations form a *polynomial ideal*, and finding a reduced *basis* for this ideal solves the word problem. And this is also what algebraists have been doing for a long time: Buchberger's algorithm and Gröbner bases are standard tools

AC word problem example, as an ideal

$$(a + b) + c = e_1$$

$$b + (c + d) = e_2$$

translate $\Rightarrow$

$$abc - e_1 = 0$$

$$bcd - e_2 = 0$$

$\Rightarrow$

$$abcd - d(abc - e_1) = -e_1d$$

$$abcd - a(bcd - e_2) = -e_2a$$

AC word problem example, as an ideal

$$e_1 d - e_2 a = 0$$

Final basis:

$$abc - e_1 = 0$$

$$bcd - e_2 = 0$$

$$e_1 d - e_2 a = 0$$

Buchberger etc

Buchberger's algorithm, and the refined F_4 and F_5 variants, are what we would like to use.

How to integrate their term-ordering fiddlyness with our flattened E-class system?

Well, Tiwari solved it in his dissertation sections on congruence closure mod AC.

It turns out E-graphs have yet to fully absorb all the insights of the term-rewriting world.

We still need AC-matching, though.

Buchberger again

We can solve not just AC, but rings, polynomials, ACU, etc, with the same techniques. So, even if there is a purely “term”-based approach (Buchberger and Tiwari, *Congruence Closure modulo AC*)

Better than Buchberger

Abelian groups (aka ACUN) can be solved with Hermite normal form.
Polynomial time! Vector spaces over a field can be solved with Gaussian elimination, which is $O(n^{2.\text{something}})$

Matching modulo AC: semantics

There is the delicate matter that **what** is considered an AC -match could affect completeness.

For instance, if we have the pattern $x + f(x)$, should we consider just the substitutions $x := t$ where $t + f(t)$ is in the E-graph, or do we have to broaden to consider this when t and $f(t)$, but not $t + f(t)$, are in the E-graph?

Matching modulo AC and Gröbner bases

Our actual representation of AC-terms to match against is not even in the form of a bunch of E-classes and E-nodes, but some incrementally-updated Gröbner basis system, which affects what “in the E-graph” means.

The important limitation: we will only find matches where the original E-graph with AC identities would.

So, there is a fairly reasonable definition of matches even for this.

Loose ends 1: group UF

There is a notion of group union find: instead of representing a bunch of equations $a = b = c$, we can represent $a = gb = h^{-1}c$. The group structure corresponds to reflexivity, symmetry, and transitivity.

This is also some form of “EMT” although a very different thing.

Some extensions to the group UF have been recently proposed, and a unifying version is still needed.

Loose ends 2: functorial UF

My proposal, that a ‘weighted UF’ is a functor from a thin groupoid to (a subgroupoid of) some other category, hasn’t been widely accepted :P

I ought to write the details down sometime to make it more convincing.

Loose ends 3: frontiers in E-graphs

- Abstract interpretation (AI), Galois connections, and E-graphs (AI is the perspective taken in the most recent reference for weighted UFs)
- SSA and E-graphs: coinduction vs induction, sea-of-nodes, representing control flow
- E-graphs all the way down in compilers. We can do register allocation directly on SSA, why not directly on E-graphs? The dream is some integer-linear-program approximate solver for extracting programs quickly from E-graphs, and an exact solver for the -O4 usecase.

- Coloured E-graphs and conditional rewriting. Coloured E-graphs ought to be a monotonic function from a poset to the equivalence relation.
- Slotted E-graphs and binding
- Non-ground rewriting. Find better identities directly rather than sloggng through proving their consequences on ground terms. Maybe if we keep a provenance of why ground terms are equal, we can automatically find interesting non-ground identities.
- Formalizing extraction as weighted automata
- Formalizing E-class analyses (ought to be not too hard)
- E-graphs for monoidal categories

- E-graphs for regular languages (they are non-algebraically-axiomatizable, in some sense)

Loose ends 4: decision procedures

- ‘Fulls seldom differ’ (decision procedure)
- Presburger arithmetic (it’s just cool)
- Mixed boolean arithmetic and automata for deciding the linear fragment (!!)
- Bitvectors and arrays and other stuff from SMT-land
- Pointer reasoning?
- Inequality and disequality reasoning?
- Quantifier elimination, the decision procedure for the reals
- Integer linear programming, Graver or Hilbert bases
- Gosper’s algorithm, aka $A=B$

- Risch's algorithm

Loose ends 5: scribbles

Various things I would like to think more about

Automata, automatic groups, other solvable T s for EM(T), other decision procedures, quantifier instantiation, faster indexing for WCOJs...